

LAPORAN TUGAS 6

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Mochamad Rio Aprilian (2410651040)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB 1

PENDAHULUAN

1.1 TUJUAN

Melalui praktikum ini, saya dapat memahami secara nyata bagaimana deadlock muncul ketika beberapa proses bersaing untuk memperoleh resource yang jumlahnya terbatas. Dengan memanfaatkan thread dan mutex dalam bahasa C, terlihat jelas bahwa kesalahan dalam pengaturan penguncian dapat membuat program berhenti total karena tidak ada proses yang dapat melanjutkan eksekusi. Praktikum ini juga memperkenalkan berbagai metode pencegahan, seperti penggunaan lock ordering, mekanisme try-lock dengan percobaan ulang, serta penerapan Banker's Algorithm untuk mengevaluasi keamanan sistem sebelum resource dialokasikan. Dengan pendekatan tersebut, mahasiswa mampu mengidentifikasi, mencegah, dan menyelesaikan deadlock secara sistematis dan efektif.

BAB II

PEMBAHASAN

2.1 Analisis Deadlock

Jalankan program `deadlock_simple.c`

2.1.1 Mengapa program mengalami deadlock?

Deadlock terjadi karena kedua thread saling menunggu resource yang sedang dikunci oleh satu sama lain, sehingga tidak ada yang bisa melanjutkan proses. Misalnya, Thread 1 terlebih dahulu mengamankan lock1, lalu berusaha memperoleh lock2 yang pada saat itu sedang dikunci oleh Thread 2. Sementara itu, Thread 2 sudah memegang lock2 dan mencoba mendapatkan lock1 yang masih dipegang oleh Thread 1. Kondisi saling bergantung ini menyebabkan kedua thread menunggu tanpa akhir, karena tidak ada yang melepaskan kunci yang dibutuhkan. Akibatnya, program terjebak dan berhenti sepenuhnya tanpa dapat melanjutkan eksekusi.

2.1.2 Kondisi deadlock apa saja yang terpenuhi?

Terdapat 4 kondisi deadlock yang terpenuhi yakni:

- Mutual exclusion
Hanya satu proses saja yang dapat menggunakan satu sumber daya dalam satu waktu.
- Hold and wait
Sebuah proses yang telah memegang setidaknya satu sumber daya menunggu untuk mendapatkan sumber daya yang telah dipegang oleh proses lain.
- No preemption
Sumber daya yang telah dipegang oleh suatu proses tidak bisa serta merta diambil oleh proses yang lain.
- Circular wait
Terjadi sebuah siklus yang Dimana setiap proses menunggu sumber daya yang dipegang oleh proses yang lain.

2.1.3 Gambar diagram resource allocation graph-nya

Thread 1 → Lock 1 → Thread 2 → Lock 2 → Thread 1 (Terus Berulang)

Dan proses tersebut terus berulang

Jika dalam bentuk hubungan:

- Ti memegang lock 1
- Ti menunggu lock 2
- T2 memegang lock 2
- T2 menunggu lock 1

Dan hubungan diatas akan mengakibatkan siklus yang merupakan indikasi deadlock.

2.2 Implementasi pencegahan

Modifikasi program deadlock_simple.c menggunakan teknik:

2.2.1 Lock ordering

```
rio_regex@DESKTOP-NVPMHQ:~$ nano deadlock_simple_lock_ordering.c
rio_regex@DESKTOP-NVPMHQ:~$ gcc deadlock_simple_lock_ordering.c -o lock_ordering -lpthread
rio_regex@DESKTOP-NVPMHQ:~$ ./lock_ordering
Thread 1: mencoba lock mutex1...
Thread 1: mutex1 terkunci.
Thread 1: mencoba lock mutex2...
Thread 1: mutex2 terkunci.
Thread 1: menjalankan critical section...
Thread 2: mencoba lock mutex1...
Thread 1: selesai.
Thread 2: mutex1 terkunci.
Thread 2: mencoba lock mutex2...
Thread 2: mutex2 terkunci.
Thread 2: menjalankan critical section...
Thread 2: selesai.
rio_regex@DESKTOP-NVPMHQ:~$ |
```

KODE C:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex1;
pthread_mutex_t mutex2;

void* thread1(void* arg) {
    printf("Thread 1: mencoba lock mutex1...\n");
    pthread_mutex_lock(&mutex1);
    printf("Thread 1: mutex1 terkunci.\n");

    printf("Thread 1: mencoba lock mutex2...\n");
    pthread_mutex_lock(&mutex2);
    printf("Thread 1: mutex2 terkunci.\n");

    printf("Thread 1: menjalankan critical section...\n");
    sleep(1);

    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    printf("Thread 1: selesai.\n");

    return NULL;
}
```

```

void* thread2(void* arg) {
    printf("Thread 2: mencoba lock mutex1...\n");
    pthread_mutex_lock(&mutex1);
    printf("Thread 2: mutex1 terkunci.\n");

    printf("Thread 2: mencoba lock mutex2...\n");
    pthread_mutex_lock(&mutex2);
    printf("Thread 2: mutex2 terkunci.\n");

    printf("Thread 2: menjalankan critical section...\n");
    sleep(1);

    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    printf("Thread 2: selesai.\n");

    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&mutex1, NULL);
    pthread_mutex_init(&mutex2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&mutex1);
    pthread_mutex_destroy(&mutex2);

    return 0;
}

```

2.2.2 Try-lock dengan retry

```

rio_regex@DESKTOP-NVPMHQ:~$ nano deadlock_simple_try_lock.c
rio_regex@DESKTOP-NVPMHQ:~$ gcc deadlock_simple_try_lock.c -o try_lock -lpthread
rio_regex@DESKTOP-NVPMHQ:~$ ./try_lock
Thread 1: mencoba lock mutex1...
Thread 1: mencoba lock mutex2...
Thread 1: semua mutex terkunci.
Thread 1: critical section...
Thread 2: mencoba lock mutex2...
Thread 1: selesai.
Thread 2: mencoba lock mutex1...
Thread 2: semua mutex terkunci.
Thread 2: critical section...
Thread 2: selesai.
rio_regex@DESKTOP-NVPMHQ:~$ |

```

KODE C:

```
#include <stdio.h>
```

```

#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex1;
pthread_mutex_t mutex2;

void* thread1(void* arg) {
    while (1) {
        printf("Thread 1: mencoba lock mutex1...\n");
        pthread_mutex_lock(&mutex1);

        printf("Thread 1: mencoba lock mutex2...\n");
        if (pthread_mutex_trylock(&mutex2) == 0) {
            printf("Thread 1: semua mutex terkunci.\n");
            break;
        } else {
            printf("Thread 1: gagal lock mutex2, retry...\n");
            pthread_mutex_unlock(&mutex1);
            usleep(100000); // 0.1 detik
        }
    }

    printf("Thread 1: critical section...\n");
    sleep(1);
    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    printf("Thread 1: selesai.\n");

    return NULL;
}

void* thread2(void* arg) {
    while (1) {
        printf("Thread 2: mencoba lock mutex2...\n");
        pthread_mutex_lock(&mutex2);

        printf("Thread 2: mencoba lock mutex1...\n");
        if (pthread_mutex_trylock(&mutex1) == 0) {
            printf("Thread 2: semua mutex terkunci.\n");
            break;
        } else {
            printf("Thread 2: gagal lock mutex1, retry...\n");
            pthread_mutex_unlock(&mutex2);
            usleep(100000); // 0.1 detik
        }
    }

    printf("Thread 2: critical section...\n");
    sleep(1);
    pthread_mutex_unlock(&mutex1);
    pthread_mutex_unlock(&mutex2);
}

```

```

printf("Thread 2: selesai.\n");
return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_mutex_init(&mutex1, NULL);
    pthread_mutex_init(&mutex2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&mutex1);
    pthread_mutex_destroy(&mutex2);

    return 0;
}

```

2.2.3 Bandingkan kedua metode tersebut

- Lock ordering memiliki kelebihan yakni lebih simple dan efisien serta tidak ada retry sedangkan kekurangannya adalah dia membutuhkan desain urutan lock yang konsisten disuluruh programnya.
- Try lock and retry memiliki kelebihan yakni fleksibel tidak membutuhkan urutan lock yang konsisten sedangkan kekurangannya adalah jika tidak efisien maka ada banyak loop retry.

2.3 Banker's algorithm

Buat program baru dengan data:

- 4 Proses
- 3 jenis resource
- Available: [5, 4, 3]
- Implementasikan request resource dan cek keamanan system

```

rio_regex@DESKTOP-NVPMSHQ:~$ nano bankers_algorithm.c
rio_regex@DESKTOP-NVPMSHQ:~$ gcc bankers_algorithm.c -o bankers_algorithm -lpthread
rio_regex@DESKTOP-NVPMSHQ:~$ ./bankers_algorithm
Available awal: 5 4 3
Safe Sequence: 0 1 2 3
Sistem aman pada kondisi awal.

Request dari proses P1: 1 0 1
Safe Sequence: 0 1 2 3
Request dapat diberikan.
Sistem tetap aman.

Request dari proses P0: 2 1 0
Safe Sequence: 0 1 2 3
Request dapat diberikan.
Sistem tetap aman.

Program selesai.
rio_regex@DESKTOP-NVPMSHQ:~$ |

```

Penjelasan:

Kode tersebut merupakan implementasi dari Banker's Algorithm yang digunakan untuk mencegah terjadinya deadlock dengan memastikan setiap permintaan resource hanya diberikan jika sistem tetap berada pada kondisi aman. Program menyimpan data tentang jumlah resource yang tersedia, kebutuhan maksimum tiap proses, serta alokasi yang sedang digunakan. Matriks need dihitung sebagai selisih antara maksimum dan alokasi saat ini. Fungsi isSafe() mengevaluasi apakah urutan eksekusi proses masih memungkinkan tanpa melampaui kapasitas resource. Sementara itu, requestResource() memproses permintaan dengan memeriksa kecukupan need dan available, melakukan alokasi sementara, lalu memvalidasi kembali keamanan sistem. Jika menyebabkan unsafe state, perubahan dikembalikan. Dalam fungsi main, program menghitung need dan menguji beberapa permintaan resource.

KODE C:

```

#include <stdio.h>
#include <stdbool.h>

#define P 4 // jumlah proses
#define R 3 // jumlah jenis resource

int available[R] = {5, 4, 3};

int maximum[P][R] = {
    {4, 3, 2},
    {2, 2, 3},
    {3, 3, 2},
    {5, 3, 3}
};

int allocation[P][R] = {
    {1, 0, 1},
    {1, 1, 2},
    {1, 0, 3},
    {0, 2, 1}
};

int need[P][R];

```



```

void calcNeed() {
    int i, j;
    for (i = 0; i < P; i++) {
        for (j = 0; j < R; j++) {
            need[i][j] = maximum[i][j] - allocation[i][j];
        }
    }
}

```

```

bool isSafe() {
    bool finish[P] = {false};
    int safeSeq[P];
    int work[R];
    int i, j, k;
    int count = 0;

    for (i = 0; i < R; i++) {
        work[i] = available[i];
    }

    while (count < P) {
        bool found = false;

        for (i = 0; i < P; i++) {
            if (!finish[i]) {
                bool canRun = true;
                for (j = 0; j < R; j++) {
                    if (need[i][j] > work[j]) {
                        canRun = false;
                        break;
                    }
                }
            }

            if (canRun) {
                for (k = 0; k < R; k++) {
                    work[k] += allocation[i][k];
                }
                safeSeq[count] = i;
                finish[i] = true;
                count++;
                found = true;
            }
        }

        if (!found) {
            return false;
        }
    }
}

```

```

printf("Safe Sequence: ");
for (i = 0; i < P; i++) {
    printf("%d ", safeSeq[i]);
}
printf("\n");

return true;
}

bool requestResource(int p, int req[]) {
    int i;

    printf("\nRequest dari proses P%d: ", p);
    for (i = 0; i < R; i++) {
        printf("%d ", req[i]);
    }
    printf("\n");

    for (i = 0; i < R; i++) {
        if (req[i] > need[p][i]) {
            printf("Request lebih besar dari need proses.\n");
            return false;
        }
    }

    for (i = 0; i < R; i++) {
        if (req[i] > available[i]) {
            printf("Resource tidak cukup. Request ditolak.\n");
            return false;
        }
    }

    for (i = 0; i < R; i++) {
        available[i] -= req[i];
        allocation[p][i] += req[i];
        need[p][i] -= req[i];
    }

    if (isSafe()) {
        printf("Request dapat diberikan.\n");
        printf("Sistem tetap aman.\n");
        return true;
    } else {
        printf("Request menyebabkan sistem tidak aman.\n");
        printf("Rollback.\n");

        for (i = 0; i < R; i++) {
            available[i] += req[i];
            allocation[p][i] -= req[i];
        }
    }
}

```

```

        need[p][i] += req[i];
    }
    return false;
}
}

int main() {

    calcNeed();

    printf("Available awal: %d %d %d\n", available[0], available[1], available[2]);

    if (isSafe()) {
        printf("Sistem aman pada kondisi awal.\n");
    } else {
        printf("Sistem tidak aman pada kondisi awal.\n");
    }

    int req1[R] = {1, 0, 1};
    requestResource(1, req1);

    int req2[R] = {2, 1, 0};
    requestResource(0, req2);

    printf("\nProgram selesai.\n");
    return 0;
}

```

2.4 Real Case

Buat simulasi deadlock untuk kasus: "Dua orang ingin transfer uang antar rekening secara bersamaan"

- Orang A: transfer dari Rekening 1 ke Rekening 2
- Orang B: transfer dari Rekening 2 ke Rekening 1
- Implementasikan deadlock prevention

```

rio_regex@DESKTOP-NVPMHQ:~$ nano deadlock_prevention.c
rio_regex@DESKTOP-NVPMHQ:~$ gcc deadlock_prevention.c -o deadlock_prevention -lpthread
rio_regex@DESKTOP-NVPMHQ:~$ ./deadlock_prevention
SIMULASI TRANSFER TANPA DEADLOCK

Saldo awal:
Rek1 = 1000
Rek2 = 2000

A mulai transfer 300 dari Rek1 ke Rek2...
B mulai transfer 500 dari Rek2 ke Rek1...
A selesai transfer.
B selesai transfer.

Saldo akhir:
Rek1 = 1200
Rek2 = 1800

PROGRAM SELESAI TANPA DEADLOCK
rio_regex@DESKTOP-NVPMHQ:~$ |

```

KODE C:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

int saldo1 = 1000;
int saldo2 = 2000;

pthread_mutex_t lock_rek1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock_rek2 = PTHREAD_MUTEX_INITIALIZER;

void lock_in_order(pthread_mutex_t *first, pthread_mutex_t *second) {
    pthread_mutex_lock(first);
    pthread_mutex_lock(second);
}

void unlock_in_order(pthread_mutex_t *first, pthread_mutex_t *second) {
    pthread_mutex_unlock(second);
    pthread_mutex_unlock(first);
}

void* transfer_A(void* arg) {
    printf("A mulai transfer 300 dari Rek1 ke Rek2...\n");

    lock_in_order(&lock_rek1, &lock_rek2);

    saldo1 -= 300;
    saldo2 += 300;
    sleep(1);

    printf("A selesai transfer.\n");

    unlock_in_order(&lock_rek1, &lock_rek2);

    return NULL;
}

void* transfer_B(void* arg) {
    printf("B mulai transfer 500 dari Rek2 ke Rek1...\n");

    lock_in_order(&lock_rek1, &lock_rek2);

    saldo2 -= 500;
    saldo1 += 500;
    sleep(1);

    printf("B selesai transfer.\n");

    unlock_in_order(&lock_rek1, &lock_rek2);
}
```

```

    return NULL;
}

int main() {
    pthread_t A, B;

    printf("SIMULASI TRANSFER TANPA DEADLOCK\n\n");
    printf("Saldo awal:\nRek1 = %d\nRek2 = %d\n", saldo1, saldo2);

    pthread_create(&A, NULL, transfer_A, NULL);
    pthread_create(&B, NULL, transfer_B, NULL);

    pthread_join(A, NULL);
    pthread_join(B, NULL);

    printf("\nSaldo akhir:\nRek1 = %d\nRek2 = %d\n", saldo1, saldo2);
    printf("\nPROGRAM SELESAI TANPA DEADLOCK\n");

    return 0;
}

```

Cara kerjanya:

- Kedua thread saling mengakses dua rekening yang berbeda.
- Jika masing-masing mengunci rekening yang berbeda duluan, deadlock berpotensi terjadi.
- Solusi lock ordering:

Semua thread harus mengambil kunci rekening dengan urutan numerik yang sama:

Rekening 1 → Rekening 2 sehingga tidak ada kondisi saling menunggu yang mengakibatkan deadlock.

Penjelasannya:

Program ini melakukan simulasi transfer dana antara dua rekening menggunakan dua thread yang berjalan secara paralel. Masing-masing thread melakukan transaksi dengan arah berbeda, sehingga keduanya berpotensi berebut akses terhadap data saldo. Untuk menghindari terjadinya deadlock, program menerapkan strategi lock ordering, yaitu setiap thread diwajibkan mengunci mutex rekening dalam urutan yang konsisten: pertama mengunci Rekening 1, lalu Rekening 2. Dengan pendekatan ini, situasi saling tunggu tidak akan muncul karena kedua thread mengikuti pola penguncian yang sama. Setelah penguncian berhasil, proses pemindahan saldo dijalankan dan mutex dilepas kembali sesuai urutan, memungkinkan seluruh operasi selesai dengan aman tanpa hambatan.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan , dapat dipahami bahwa deadlock muncul apabila sejumlah kondisi spesifik terpenuhi, terutama ketika proses saling menunggu resource yang tidak pernah dilepaskan oleh proses lain. Melalui simulasi, terlihat bahwa tanpa mekanisme pengelolaan penguncian yang benar, program mudah masuk ke kondisi buntu dan tidak dapat melanjutkan eksekusi. Penggunaan teknik seperti lock ordering terbukti efektif mencegah deadlock dengan memastikan semua thread mengikuti urutan penguncian yang sama. Di sisi lain, Banker's Algorithm menyediakan metode analisis yang terstruktur untuk menilai apakah suatu permintaan resource aman dipenuhi, sehingga sistem dapat terhindar dari keadaan yang berpotensi menyebabkan deadlock.